

```
# FIND-S Algorithm
# -----

import pandas as pd

# Sample training data
data = {
    'Sky': ['Sunny', 'Sunny', 'Rainy', 'Sunny'],
    'AirTemp': ['Warm', 'Warm', 'Cold', 'Warm'],
    'Humidity': ['Normal', 'High', 'High', 'High'],
    'Wind': ['Strong', 'Strong', 'Strong', 'Weak'],
    'Water': ['Warm', 'Warm', 'Warm', 'Cool'],
    'Forecast': ['Same', 'Same', 'Change', 'Change'],
    'EnjoySport': ['Yes', 'Yes', 'No', 'Yes']
}

df = pd.DataFrame(data)

print("Training Data:")
print(df)
print()

# FIND-S implementation
def find_s(data):
    # Remove the target column
    attributes = data.iloc[:, :-1].values
    target = data.iloc[:, -1].values

    # Start with most specific hypothesis
    hypothesis = ['0'] * attributes.shape[1]

    for i in range(len(data)):
        # Consider only positive examples
        if target[i] == 'Yes':
            for j in range(len(hypothesis)):
                # First positive example
                if hypothesis[j] == '0':
                    hypothesis[j] = attributes[i][j]

                # If values differ, generalize to '?'
                elif hypothesis[j] != attributes[i][j]:
                    hypothesis[j] = '?'

    return hypothesis

hypothesis = find_s(df)

print("Most Specific Hypothesis:")
print(hypothesis)
```

```
Training Data:
   Sky AirTemp Humidity  Wind Water Forecast EnjoySport
0  Sunny    Warm   Normal Strong  Warm     Same        Yes
1  Sunny    Warm    High Strong  Warm     Same        Yes
2  Rainy    Cold    High Strong  Warm   Change        No
3  Sunny    Warm    High  Weak  Cool   Change        Yes
```

```
Most Specific Hypothesis:
['Sunny', 'Warm', '?', '?', '?', '?']
```

```
import pandas as pd
import itertools

# Read training data from CSV
data = pd.read_csv("training_data.csv")

print("Training Data:")
print(data)
print()

# -----
# Function to check whether a hypothesis covers example
# -----
```

```

def covers(hypothesis, example):

    for h, x in zip(hypothesis, example):

        # '?' means any value is accepted
        if h == '?':
            continue

        # Specific value must match
        if h != x:
            return False

    return True

# -----
# Generate all possible hypotheses
# -----
attributes = list(data.columns[:-1])
target = data.columns[-1]

domains = []

for attribute in attributes:
    values = list(data[attribute].unique())
    domains.append(values)

# Add '?' to every attribute domain
domains_with_question = []

for values in domains:
    domains_with_question.append(values + ['?'])

# Generate Cartesian product
all_hypotheses = list(
    itertools.product(*domains_with_question)
)

print("Total possible hypotheses:", len(all_hypotheses))
print()

# -----
# Find all hypotheses consistent with training data
# -----
consistent_hypotheses = []

for hypothesis in all_hypotheses:

    consistent = True

    for _, row in data.iterrows():

        example = row.iloc[:-1].tolist()
        actual_class = row.iloc[-1]

        prediction = covers(hypothesis, example)

        # Positive example must be covered
        if actual_class == 'Yes' and not prediction:
            consistent = False
            break

        # Negative example must NOT be covered
        if actual_class == 'No' and prediction:
            consistent = False
            break

    if consistent:
        consistent_hypotheses.append(hypothesis)

# -----
# Find Specific Boundary (S)
# -----

```

```

def is_more_specific(h1, h2):
    """
    Returns True if h1 is more specific than h2.
    """
    strictly_more_specific = False
    for a, b in zip(h1, h2):
        if a == b:
            continue
        # '?' is more general
        if b == '?':
            strictly_more_specific = True
        else:
            return False
    return strictly_more_specific

S = []

for h in consistent_hypotheses:
    is_most_specific = True
    for other in consistent_hypotheses:
        if h != other and is_more_specific(other, h):
            is_most_specific = False
            break
    if is_most_specific:
        S.append(h)

# -----
# Find General Boundary (G)
# -----
G = []

for h in consistent_hypotheses:
    is_most_general = True
    for other in consistent_hypotheses:
        if h != other and is_more_specific(h, other):
            is_most_general = False
            break
    if is_most_general:
        G.append(h)

# -----
# Display Results
# -----

print("=====")
print(" CANDIDATE-ELIMINATION RESULTS")
print("=====")

print("\nAttributes:")
print(attributes)

print("\nNumber of Consistent Hypotheses:")
print(len(consistent_hypotheses))

print("\nALL CONSISTENT HYPOTHESES (VERSION SPACE):")

for i, h in enumerate(consistent_hypotheses, 1):
    print(i, ":", h)

```

```

print("\nSPECIFIC BOUNDARY (S):")

for h in S:
    print(h)

print("\nGENERAL BOUNDARY (G):")

for h in G:
    print(h)

```

Training Data:

	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
0	Sunny	Warm	Normal	Strong	Warm	Same	Yes
1	Sunny	Warm	High	Strong	Warm	Same	Yes
2	Rainy	Cold	High	Strong	Warm	Change	No
3	Sunny	Warm	High	Weak	Cool	Change	Yes

Total possible hypotheses: 729

```

=====
CANDIDATE-ELIMINATION RESULTS
=====

```

Attributes:
['Sky', 'AirTemp', 'Humidity', 'Wind', 'Water', 'Forecast']

Number of Consistent Hypotheses:
3

ALL CONSISTENT HYPOTHESES (VERSION SPACE):

```

1 : ('Sunny', 'Warm', '?', '?', '?', '?')
2 : ('Sunny', '?', '?', '?', '?', '?')
3 : ('?', 'Warm', '?', '?', '?', '?')

```

SPECIFIC BOUNDARY (S):
('Sunny', 'Warm', '?', '?', '?', '?')

GENERAL BOUNDARY (G):
('Sunny', '?', '?', '?', '?', '?')
('?', 'Warm', '?', '?', '?', '?')

```

# ID3 Decision Tree Algorithm
# -----

import pandas as pd
import math
from collections import Counter

# Creating the Play Tennis dataset
data = {
    'Outlook': [
        'Sunny', 'Sunny', 'Overcast', 'Rain',
        'Rain', 'Rain', 'Overcast', 'Sunny',
        'Sunny', 'Rain', 'Sunny', 'Overcast',
        'Overcast', 'Rain'
    ],
    'Temperature': [
        'Hot', 'Hot', 'Hot', 'Mild',
        'Cool', 'Cool', 'Cool', 'Mild',
        'Cool', 'Mild', 'Mild', 'Mild',
        'Hot', 'Mild'
    ],
    'Humidity': [
        'High', 'High', 'High', 'High',
        'Normal', 'Normal', 'Normal', 'High',
        'Normal', 'Normal', 'Normal', 'High',
        'Normal', 'High'
    ],
    'Wind': [
        'Weak', 'Strong', 'Weak', 'Weak',
        'Weak', 'Strong', 'Strong', 'Weak',
        'Weak', 'Weak', 'Strong', 'Strong',
        'Weak', 'Strong'
    ],

```

```
....'PlayTennis':·[
.....'No',·'No',·'Yes',·'Yes',
.....'Yes',·'No',·'Yes',·'No',
.....'Yes',·'Yes',·'Yes',·'Yes',
.....'Yes',·'No'
....]
}

df = pd.DataFrame(data)

print("PLAY TENNIS DATASET")
print("-----")
print(df)
```

PLAY TENNIS DATASET

	Outlook	Temperature	Humidity	Wind	PlayTennis
0	Sunny	Hot	High	Weak	No
1	Sunny	Hot	High	Strong	No
2	Overcast	Hot	High	Weak	Yes
3	Rain	Mild	High	Weak	Yes
4	Rain	Cool	Normal	Weak	Yes
5	Rain	Cool	Normal	Strong	No
6	Overcast	Cool	Normal	Strong	Yes
7	Sunny	Mild	High	Weak	No
8	Sunny	Cool	Normal	Weak	Yes
9	Rain	Mild	Normal	Weak	Yes
10	Sunny	Mild	Normal	Strong	Yes
11	Overcast	Mild	High	Strong	Yes
12	Overcast	Hot	Normal	Weak	Yes
13	Rain	Mild	High	Strong	No